

How devtooling resources get into LLM training data

A practical, sourced guide to how a developer-tools ecosystem (a content website, documentation, blog posts, and code) reaches the training corpora of frontier models, which content actually makes it, and how to optimize and verify each path.

Research note · last updated 2026-06-20 · every major fact links a primary source

CONTENTS

- | | |
|--|---|
| 1. The mental model: two gates and four channels | 7. Other channels worth knowing |
| 2. Key terms | 8. A note on llms.txt |
| 3. How models acquire knowledge of a language or framework | 9. What Anthropic discloses |
| 4. Gate 1: Common Crawl and web collection | 10. Guidance: how to optimize, by channel |
| 5. Gate 2: the quality filters on web data | 11. Guidance: how to check |
| 6. Gate 1 + 2 for code: The Stack and code corpora | 12. Caveats and what is contested |
| | 13. Sources |

It is written for the Ruby/Rails discoverability work, but the mechanics are general. Every major factual claim links to a primary source. Findings labeled **(measured)** come from our own probes in this repo and are proxies, not ground truth (see [Caveats](#)).

The mental model: two gates and four channels

A page or file becomes training data only if it clears **two independent gates**, through some **channel**:

1. **Gate 1, inclusion in a corpus source.** The content has to be physically collected, by a web crawl (Common Crawl and lab crawlers) or from public code hosting (GitHub via Software Heritage).
2. **Gate 2, surviving the filter.** Corpus builders then drop most of what they collect with quality, dedup, and safety filters. Modern web pipelines keep on the order of 10% of raw crawl (RefinedWeb removed ~90%).

Inclusion is **necessary but not sufficient**. Being crawlable does not mean being trained on.

The four channels carry very different content and clear the gates very differently:

Channel	Corpus it feeds	Gate 1 (collection)	Gate 2 (filter)
Content website	Web corpora (C4, RefinedWeb, FineWeb)	Common Crawl / lab crawl	Quality + safety heuristics, then model-based quality classifiers
Documentation (rendered)	Web corpora	Common Crawl / lab crawl	Same web filters (hostile to reference/code)
Blog posts	Web corpora	Common Crawl / lab crawl	Same web filters (rewards tutorial prose)
Code + docs-in-repos	Code corpora (The Stack, StarCoder)	GitHub, public + permissively licensed only	License + dedup, no prose-quality classifier

The single most important consequence: **code corpora do not apply the web quality filter**. A library's README, examples, and the Markdown its docs are built from can reach code-trained models even when the rendered docs site fails the web gates entirely.

Key terms

Plain definitions of the jargon used below, with the actionable takeaway for each.

Collection (Gate 1)

- **Common Crawl:** a free, public archive of web pages, re-crawled roughly monthly, that is the raw material for most open web training corpora. Being in it is the entry ticket for the web channel.

- **Centrality (harmonic centrality):** a link-popularity score. The more other pages link to you, directly or through short chains of links, the higher your centrality. Crawlers use it to decide which sites to fetch and how deeply. *Actionable: earn backlinks and add internal links to raise it.*
- **Crawl budget:** the capped number of URLs a crawler will fetch from one domain in a pass. Higher centrality buys a bigger budget. *Actionable: reduce click-depth so important pages fit the budget.*
- **Recall (coverage):** the share of a site's real pages that actually made it into a crawl. Common Crawl's per-domain recall is partial (we measured 27 to 37%). *Actionable: verify it, do not assume full coverage.*
- **Software Heritage:** the universal archive of public source code that the code corpora are built from.
- **CDX index:** Common Crawl's queryable index of captured URLs. How you check whether a page was crawled.
- **robots.txt:** the file that tells crawlers what they may fetch. The one lever you fully control over whether ClaudeBot , CCBot , and GPTBot collect your site. *Actionable: do not block them if you want in.*
- **sitemap.xml:** a list of your URLs offered to crawlers. A discovery hint, not a guarantee of inclusion.

Filtering (Gate 2)

- **Corpus:** a large, curated dataset of text or code that a model trains on (C4, FineWeb, The Stack).
- **Deduplication:** removing near-identical text so a model does not over-learn repeats, done with MinHash (fuzzy matching) and exact-substring matching. Boilerplate repeated across your pages collapses to one copy.
- **Heuristic filter:** a cheap hand-written rule (for example, "drop any page containing a { ") used to clean a corpus. Blunt and prone to false positives on technical content.
- **Quality classifier:** a small model that rates each page's "educational value" and drops low scorers. FineWeb-Edu keeps pages scoring 3 or higher on a 0 to 5 scale. *Actionable: write explanatory prose, then score it.*
- **Permissive vs. copyleft license:** permissive licenses (MIT, Apache) allow reuse with few conditions and are kept by code corpora; copyleft (GPL) is largely excluded. *Actionable: license libraries permissively.*
- **Opt-out:** a mechanism (for example, BigCode's "Am I in the Stack?") to have your code removed from a dataset.

How models learn

- **Pretraining vs. fine-tuning:** pretraining is the long first phase where a model reads trillions of tokens and absorbs general knowledge, this is where it learns whether it "knows" Ruby. Fine-tuning is a shorter later phase that shapes behavior. Getting into pretraining is the goal here.
- **Token:** the unit a model reads, roughly three-quarters of a word. The quality classifier scores only the first ~512 tokens (~380 words) of a page. *Actionable: front-load substance into the opening.*
- **Representation, low- vs. high-resource:** how much of a language exists in training data. High-resource (Python) yields strong models; low-resource (Ruby, relative to Python) yields weaker ones unless boosted.
- **pass@k:** a code-benchmark metric, the probability that at least one of k generated solutions passes the tests. Higher representation in training tends to raise it.

- **Synthetic / distillation data:** training text generated by another model rather than scraped. Labs use it to lift low-resource languages beyond their organic data share.
 - **RAG and MCP:** inference-time retrieval (Retrieval-Augmented Generation; Model Context Protocol). They fetch information at request time and do not add to training. Useful for runtime accuracy, not inclusion.
 - **llms.txt:** a proposed file pointing LLMs to clean versions of your content. As of 2026, no major AI system uses it for training or inference.
-

How models acquire knowledge of a language or framework

Capability on a language tracks its **representation** (data share) in pretraining. Code LLMs do well on high-resource languages (Python, JavaScript, Java) that are abundant in training data and [struggle on low-resource ones](#) (OCaml, Racket, and others), and performance on the [MultiPL-E benchmark](#) rises with a language's frequency in the corpus.

Two nuances that matter for strategy:

- **Volume is not destiny.** [MultiPL-T / knowledge-transfer work](#) shows semi-synthetic data generated for a low-resource language, then used for fine-tuning, beats training longer on the scarce natural data. Labs actively close gaps this way, so a stack can be "taught" beyond its organic data share.
 - **Pretraining presence vs. inference-time retrieval are different levers.** Whether a model *knows* your framework is set at pretraining. Retrieval tools (RAG, MCP, `llms.txt`, content negotiation) only change what a model can *look up at runtime*; they do not add to training (see [llms.txt](#)).
-

Gate 1: Common Crawl and web collection

[Common Crawl](#) is the backbone of every open web corpus (C4, RefinedWeb, FineWeb). It is **not a copy of the web**. Per the [FAccT'24 analysis](#) and [Mozilla's writeup](#):

- Selection is by **harmonic centrality**: the more a domain is linked to, the higher its score, and "the higher the domain's score, the more URLs" are sampled from it. Within a domain, the score is projected to all URLs and the highest-ranking ones are kept.
- It samples from a CrawlDB of ~25B URLs and fetches **3-5B URLs per monthly crawl**, with **~50% being URLs re-fetched from previous crawls**.
- It **never collects full copies of domains** ("There are no complete copies of Wikipedia"), and the chief engineer states plainly that the claim it "contains the entire web" is "absolutely not true."

- It is HTML-only in most cases (no CSS, usually no images), and **US-infrastructure / English-skewed**.

What this means for a domain. High-centrality hub pages (home, section indexes) are reliably included; deep, new, or weakly-linked leaf pages are sampled at a fraction and accumulate slowly across monthly crawls. Sitemaps and `llms.txt` do not force inclusion; **internal links and external backlinks (centrality) do**. Newer crawl-efficiency research like [Craw4LLM](#) reinforces that pretraining-oriented crawlers prioritize by influence, not coverage.

(measured) For `evilmartians.com`, 221 of 600 live pages (37%) were in crawl `CC-MAIN-2026-21`; every main-nav hub was present, while deep Chronicles posts were sampled ~39%. For `docs.anycable.io`, 34 of 125 (27%). Crawled and missing pages were statistically identical in quality (below), so the gap is crawl reach, not page quality.

How to check Common Crawl inclusion

Query the [Common Crawl index \(CDX\) server](#) (no API key):

```
# all captured URLs under a domain in one crawl
https://index.commoncrawl.org/CC-MAIN-2026-21-index?url=docs.anycable.io/*&output=json
```

Tools: [cdx-index-client](#). (Note: the index is IP rate-limited.) This repo automates a live-site-vs-CC diff in `scripts/coverage_details.rb`.

Gate 2: the quality filters on web data

Once collected, web text is aggressively filtered. The lineage:

C4 (2019, the T5 corpus): cheap heuristics

[C4](#) applied hand-written rules. Verbatim from the T5 paper, the ones that bite developer content:

- Keep only **lines** ending in terminal punctuation; discard pages with **fewer than 3 sentences** and keep only lines with **at least 5 words**. (Many secondary summaries swap these to "5 sentences / 3 words"; the primary source says 3 sentences / 5 words.)
- Drop any **page** containing a word from the ["Dirty, Naughty, Obscene or Otherwise Bad Words" list](#), which [disproportionately removes minority-dialect and LGBTQIA+ text](#).

- **Drop any page that contains a curly brace** `{`. Verbatim: "Since the curly bracket '{' appears in many programming languages... but not in natural text, we removed any pages that contained a curly bracket." This is **page-level**: one code sample removes the whole page. (The line-level rules are different ones: the terminal-punctuation rule, the "Javascript" warning-line rule, and "terms of use"/"cookie policy" boilerplate lines.)

C4's filters are openly hostile to technical content. Caveat: **C4 is a 2019 corpus**. It shaped T5 and is part of [The Pile](#), but modern pipelines use better filters and do not blanket-drop braces, so treat the curly-brace rule as an instructive historical example, not a current universal rule.

RefinedWeb and FineWeb: dedup plus model-based quality

[RefinedWeb](#) (Falcon) showed properly filtered + deduplicated web data alone can beat curated corpora, and **removed ~90%** of raw content via MassiveWeb-style heuristics, MinHash fuzzy dedup, and exact-substring dedup. [FineWeb](#) refined this further and added a model-based educational-quality filter.

FineWeb-Edu classifier: the model-based gate (our proxy)

[FineWeb-Edu](#) trained a [classifier](#) on **450k Llama-3-70B annotations** scoring "educational value" 0-5; the corpus keeps documents scoring ≥ 3 (a binary threshold with F1 82%), retaining $\sim 1.3T$ tokens. Crucially, it was **deliberately tuned toward grade/middle-school knowledge and away from "highly technical pages like arXiv,"** so its bias against dense technical and reference content is *by design*. This is the closest open proxy for "would a modern web corpus keep this page."

(measured) What this does to developer content (FineWeb-Edu, scoring the first ~ 512 tokens, which is all the classifier reads):

- Of all 598 `evilmartians.com` pages, **1 cleared ≥ 3** ; 0 of 221 already-crawled pages did. AnyCable docs: **0 of 117**. Avg scores ~ 1.5 (blog/marketing) and ~ 1.8 (reference docs).
- **Rewriting the lead into genuine teaching prose is the lever, not de-boilerplating.** A conceptual tutorial rose from 2.43 to 3.19 with a realistic "define the terms, say what you'll learn" intro, and 3.54 fully rewritten. De-boilerplating alone did nothing (and *lowered* a docs page, because it exposed more dense reference text in the scored window).
- **Genre caps the ceiling.** Reference docs plateaued at 2.83 even when fully rewritten; a benchmark war-story post topped out at 1.88. Only tutorial/explanatory prose reliably clears 3.

Gate 1 + 2 for code: The Stack and code corpora

Code models learn from GitHub-derived corpora that **bypass the web quality gate entirely**.

[The Stack v2](#) / [StarCoder2](#): 3.28B files from 104M GitHub repos via the [Software Heritage](#) graph, 600+ languages, 67.5TB raw / 32.1TB deduped. [StarCoder2 trained on 3.3-4.3T tokens](#). Two rules govern what gets in:

- **License filtering: permissive or no-license only.** Copyleft (GPL) code is largely excluded. So a repo's license affects training eligibility.
- **Opt-out exists:** the ["Am I in The Stack?"](#) tool + removal requests; [1,561 repos were removed before a training run](#).

Critically, **The Stack includes documentation file types, not just executable code.** The Stack kept [~254 GB of Markdown](#) (more than HTML) precisely because Markdown carries code documentation. So:

- A library's **README** is often the single most-trained artifact about it.
- The **Markdown/RST source that a docs site is built from** is in code corpora independent of whether the rendered site is in Common Crawl or passes FineWeb-Edu.
- Runnable `examples/` and inline doc comments enter code corpora as code.

The catch: this only applies to **public, permissively-licensed repos**. A private docs/site repo contributes nothing here.

For scale context, in [the original StarCoder/The Stack](#), Ruby was a mid/low-resource language by volume (single-digit GB after filtering), far below Python/Java/JS.

Other channels worth knowing

- **Stack Overflow / Q&A** is high-value training data and is now **licensed, not freely scraped**: [OpenAI](#) and [Google/Gemini](#) signed data deals in 2024. Public [Stack Exchange data dumps](#) are CC-BY-SA. Good Q&A about your tool is trainable, but increasingly via paid pipelines.
- **Synthetic / distillation** data is a growing share of code training (see MultiPL-T above); being well-represented in the sources models distill *from* propagates indirectly.

A note on llms.txt

`llms.txt` does **not** get you into training. Google's John Mueller states [no AI system currently uses it](#) and server logs show crawlers do not even fetch it; Google [compares it to the defunct keywords meta tag](#). It is a proposed

inference-time hint for agents, unrelated to corpus inclusion. **(measured)** stripping the "Are you an LLM? read the Markdown" banner from a page did not raise its quality score.

What Anthropic discloses

Anthropic does **not** publish a data card, source breakdown, filtering heuristics, or quality classifier for Claude. What is public:

- It crawls the public web with `ClaudeBot` (plus `Claude-User` and `Claude-SearchBot`), and **respects robots.txt**; blocking `ClaudeBot` excludes your future content from training.
- High-level data sources stated elsewhere: public web, licensed third-party data, opt-in user data, and data from contractors/synthetic generation.

So the open classifiers here are **proxies**, not Anthropic's filter. The transferable, evidence-backed conclusion is structural: documented web pipelines gate on educational-prose quality and centrality-limited crawling, and any lab building on Common Crawl + quality filtering inherits those biases to some degree. The one lever site owners definitely control for Anthropic is `robots.txt` (do not block `ClaudeBot` if you want inclusion).

Guidance: how to optimize, by channel

Code + docs-in-repos (highest leverage, bypasses the quality gate):

- Keep the library's repo **public and permissively licensed** (MIT/Apache); copyleft risks exclusion from The Stack.
- Invest in the **README** and a runnable `examples/` directory; these are what code models learn from.
- Keep docs **source** as rich Markdown in a public repo (full prose, not just terse reference), since the Markdown itself is the trained artifact.

Blog posts (can clear the web quality gate):

- Write **tutorial/explanatory** posts; they clear FineWeb-Edu ≥ 3 . Benchmark/announcement/opinion posts structurally do not, do not over-invest in optimizing those for quality.
- **Lead with teaching**: first ~380 words should state what the reader will learn and define key terms in full sentences (only the lead is scored). This is good editing, not gaming.

- Drive **backlinks** (newsletters, aggregators, cross-posts to public, crawlable sites like dev.to) so Common Crawl actually reaches the post; new deep posts are otherwise slow to be crawled.

Documentation (rendered site):

- Fix Common Crawl reach first: `sitemap.xml` , **301 redirects** for moved URLs, internal linking, backlinks. Without inclusion, quality is moot.
- Convert conceptual/guide pages to lead-first lessons (raises quality scores meaningfully); accept that pure API/reference pages will not clear the bar and rely on the GitHub/code channel for those.
- Expand or merge **thin pages** (sub-500-char pages score near zero).

Content website (marketing/landing):

- Lowest training value (marketing prose scores lowest). Treat as a centrality/backlink hub that helps *other* pages get crawled, not as trainable content itself.

Everywhere:

- Do not block `CCBot` , `ClaudeBot` , `GPTBot` , `Google-Extended` in `robots.txt` if you want inclusion.
- Do not rely on `llms.txt` for training.

Guidance: how to check

Question	How	Tool
Is it in Common Crawl?	CDX query <code>url=domain/*</code> per crawl	index.commoncrawl.org , <code>scripts/coverage_details.rb</code>
Would the quality filter keep it?	Score with FineWeb-Edu (≥ 3); remember only ~512 tokens are read	classifier , <code>scripts/quality.py</code>
Is the code in The Stack?	Check repo public + permissive license	"Am I in the Stack?"
Are crawlers allowed?	Check <code>robots.txt</code> for <code>CCBot</code> / <code>ClaudeBot</code> / <code>GPTBot</code>	server logs, <code>robots.txt</code>

Caveats and what is contested

- **Proxies, not ground truth.** FineWeb-Edu and the C4 rules are *open, documented* filters. Frontier labs (Anthropic, OpenAI, Google) do not disclose theirs. Our scores show what *representative open pipelines* do, not what Claude's filter does.
 - **Single-sample, lead-only scoring.** Our quality numbers are single runs; differences within ± 0.1 -0.3 are noise. Only the first ~ 512 tokens are scored, so the lead dominates.
 - **Era matters.** C4's curly-brace rule is from 2019; modern corpora filter differently. Do not assume any one historical rule is current practice.
 - **Correlation, not mechanism, for representation $\rightarrow$ capability.** Data share correlates with capability, but synthetic data and fine-tuning can move a low-resource language independently.
 - **Inclusion $\neq$ influence.** Even trained-on content is deduplicated and weighted; presence does not guarantee the model learned it.
-

Sources

- Common Crawl, critical analysis: [Baack, FAccT'24](#) · [Mozilla writeup](#) · [CC index](#) · [Craw4LLM](#)
- C4 / T5: [Raffel et al. \(T5\)](#) · [Documenting C4 \(Dodge et al.\)](#)
- RefinedWeb: [Penedo et al.](#)
- FineWeb / FineWeb-Edu: [Penedo et al.](#) · [dataset](#) · [classifier](#)
- The Stack / StarCoder: [StarCoder2 + The Stack v2](#) · [The Stack v1](#) · [StarCoder \(orig\)](#) · [Am I in the Stack?](#)
- Low-resource languages: [MultiPL-E](#) · [Knowledge transfer / MultiPL-T](#)
- Stack Overflow deals: [OpenAI](#) · [TechCrunch](#)
- Anthropic: [ClaudeBot / crawling docs](#) · [Search Engine Land](#)
- llms.txt: [Google does not endorse](#) · [SEJ: like keywords meta tag](#)

Last updated 2026-06-20. Measured findings are reproducible via `scripts/quality.py` and `scripts/coverage_details.rb` in this repo.

Generated from `how-devtooling-gets-into-training-data.md`. Findings marked "(measured)" are reproducible via `scripts/quality.py` and `scripts/coverage_details.rb` in this repo.